



UNIVERSITY OF
WATERLOO

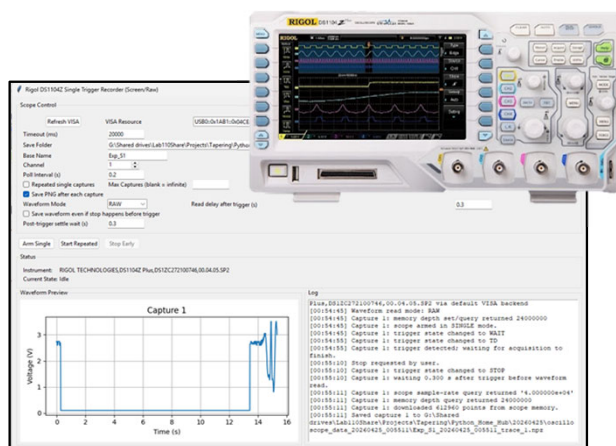


AI assisted tool building

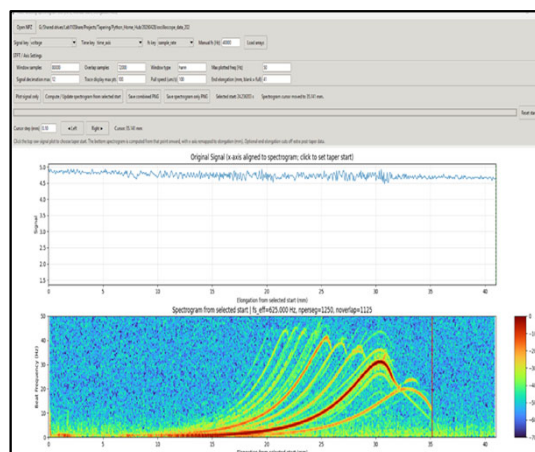
Chen Jie-En

Chen Jie-En, Wen-Ting Wang, Chun-Chia Chen, Shayne Bennetts

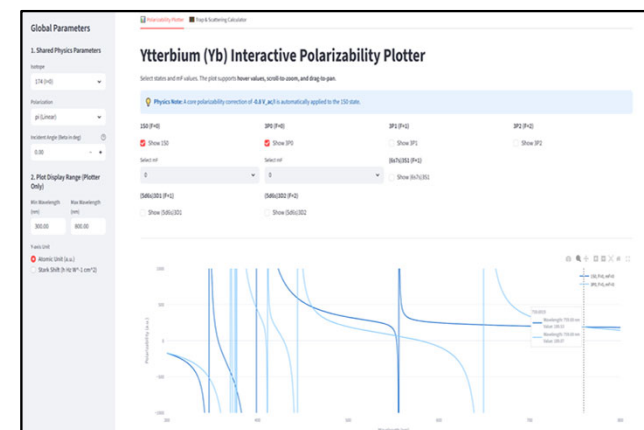
IAMS Yb Lab 110



Data Acquisition



Analysis Tools



Web Tools



3 new lab tools “vibe coded” in minutes



2 new tools to characterize our nanofibers

1. Data acquisition from an oscilloscope
2. Short time fourier transform (STFT) data analysis tool

+ A cold atom physics webapp

1. Interactive polarizability plotter and scattering rate / trap depth calculator



Gemini



Claude

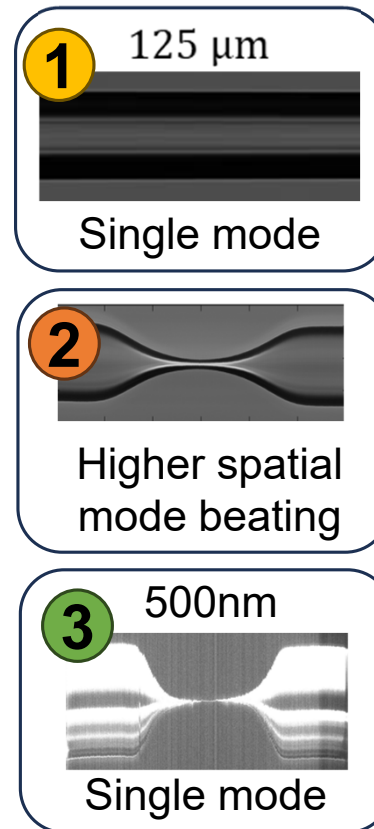
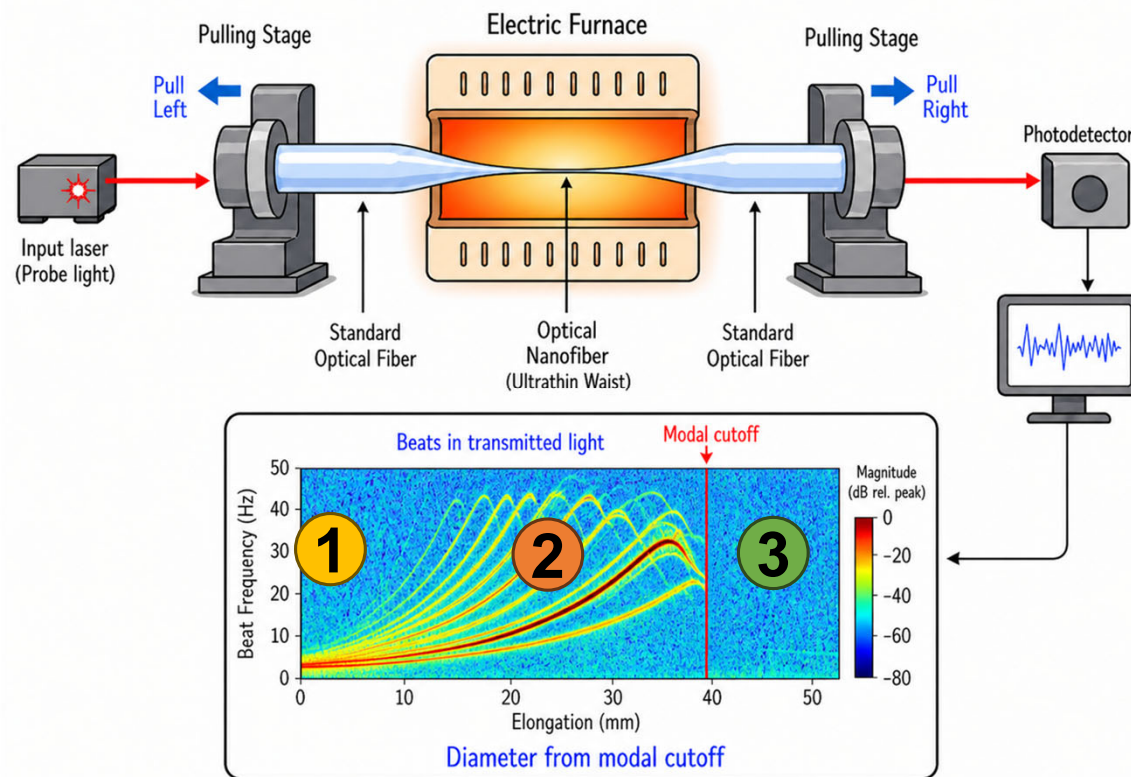


ChatGPT



Goal: Measure the diameter of nanofibers?

- Record light transmission during tapering
- Fourier transform to see beat frequency changes through time

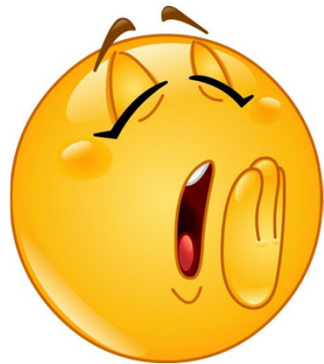
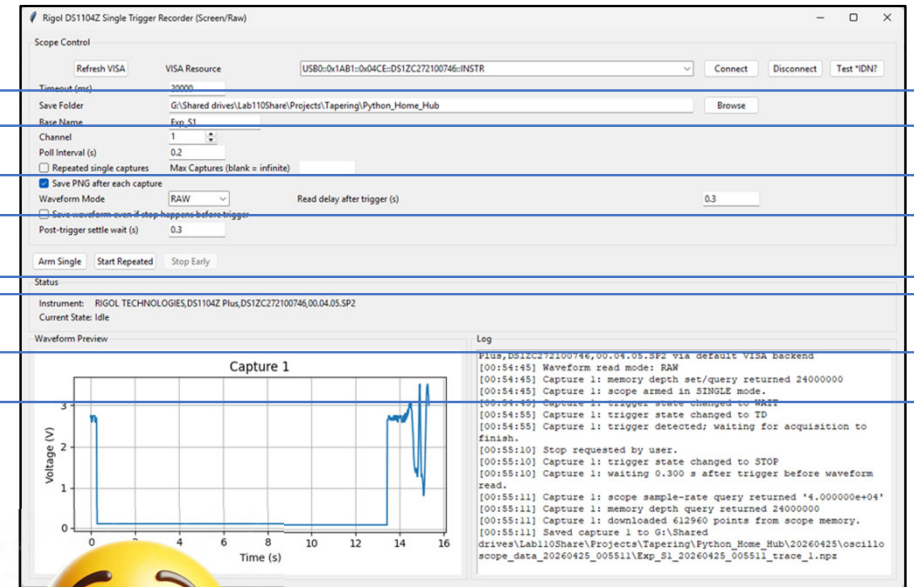




Step 1: Data acquisition

Old way:

Ask GPT to make GUI to download data over USB port



~30 minutes per trace



<30 seconds per trace

 ChatGPT



How to prompt:

1. Describe the problem (Be specific)

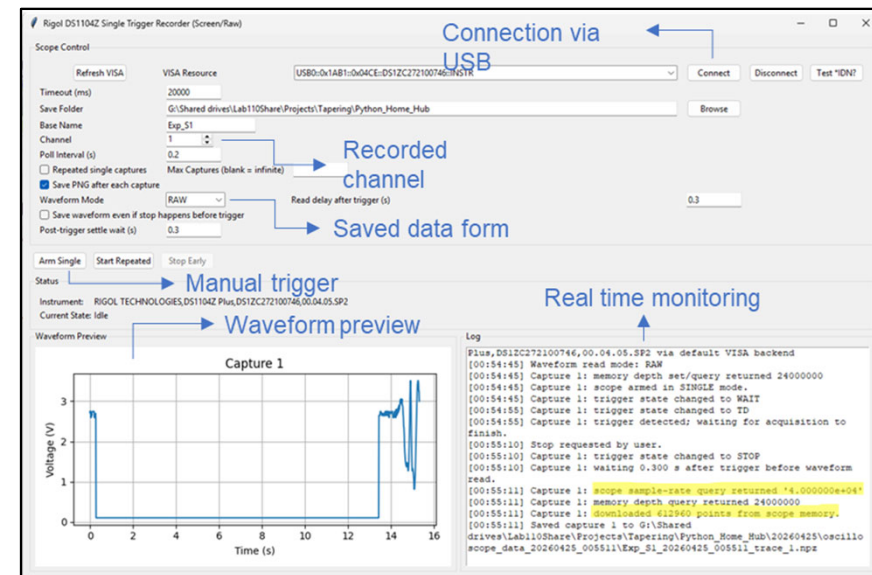
→ Rigol DS1104Z, 40k samples/s (24M memory), save file(npz,csv)

2. Define the task (Ask AI to make GUI that downloads, saves and displays)

→ easy to test and check (different function of button...)

3. Check/validate the result:

- Compare output VS oscilloscope.
- CSV file confirms 40k points/sec





Step 2: Sliding window FFT analysis



Prompt:

Build a GUI-based STFT analysis tool for time-series data with mode evolution tracking. First propose the overall design (layout, modules, and libraries) without writing full code. The tool should load data, allow adjustable STFT parameters, display the raw signal with an interactive start-time cursor, compute STFT from the selected point, and visualize a spectrogram (frequency vs time/elongation, dB) with basic mode analysis such as ridge tracking and cutoff identification.

Tips for prompting...

- **Always ask for a GUI**

Simplifies workflow; no manual code edits needed.

- **Start Broad**

Let AI suggest/plan the required functions then check them.

- **Do it Step-by-Step**

Add features iteratively to prevent breaking the code.

- **Lock it Down**

Tell the AI to only change specific sections or ask for confirmation

...



Result:



Open NPZ G:/Shared drives/Lab110Share/Projects/Tapering/Python_Home_Hub/20250428/oscilloscope_data_202 Load Data File

Signal key voltage Time key time_axis fs key sample_rate Manual fs (Hz) 40000 Load arrays

STFT / Axis Settings

Window samples 80000 Overlap samples 72000 Window type hann Max plotted freq (Hz) 50 Parameter Settings

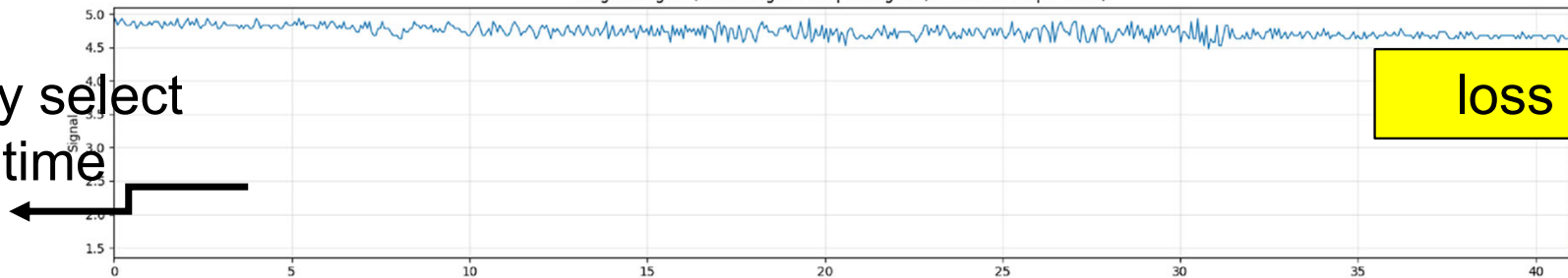
Signal decimation max 12 Trace display max pts 100 Pull speed (um/s) 100 End elongation (mm, blank = full) 41

Plot signal only Compute / Update spectrogram from selected start Save combined PNG Save spectrogram only PNG Selected start: 24.236203 s Spectrogram cursor moved to 35.141 mm.

Cursor step (mm) 0.10 Left Right Cursor: 35.141 mm

Click the top raw-signal plot to choose taper start. The bottom spectrogram is computed from that point onward, with x-axis remapped to elongation (mm). Optional end elongation cuts off extra post-taper data.

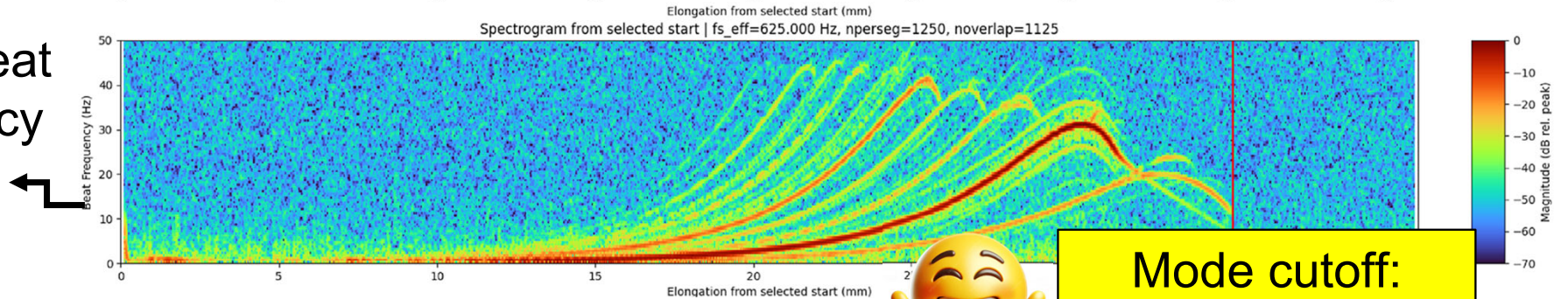
Manually select start time



loss <5%



Mode beat frequency



Mode cutoff: 488nm diameter





A webapp for atomic physics properties



The problem...my PIs kept asking:

- What's the magic wavelength of states A & B?
- What is the trap depth / scattering rate for state A at wavelength C?

What I used to do
(evaluate using my python script):

```
polarizationplot.py 2 plotting
9 def plotting(state1:str, state2:str, m1, m2, lam_in, lam_en, I, p, beta=0):
117     plt.xlabel("Wavelength (nm)")
118     plt.ylabel("V_sc/I (h Hz W^-1 cm^2)")
119     plt.xlim(lam_in, lam_en)
120     plt.ylim(-40, 40)
121     plt.title("Polarizability vs Wavelength")
122     plt.grid(True)
123     plt.legend()
124
125 # Save figure
126 filename_png = os.path.join(results_dir, f"scatter_rate_{state1}.png")
127 plt.savefig(filename_png, dpi=300, format='png')
128 plt.close()
129 print(f"Figure saved as: {filename_png}")
130
131 plotting("1S0", "3P0", 0, 0, 400, 800, 0, 0)
132
133
134
135
136
137
138
139
140
141
142 def intensity(power_mW, waist_um):
143     power_W = power_mW * 1e-3
144     waist_m = waist_um * 1e-6
145     I_0 = 2 * power_W / (np.pi * waist_m**2)
146     return I_0
147
148 I = intensity(0.02, 100)
149 rate = single_transition_scattering(2*np.pi*80*1e6, I, 2*np.pi*180*1e3)
150 print(rate)
```

Solution:

- Level 1: Ask AI to make a GUI wrapper
- Level 2: Ask AI to make a webapp





How to build an app with AI



Framework: Ask for AI recommendations → Streamlit (GitHub integrated)

Integration: Feed original code → Generate initial web app

Iteration: Prompt for desired functions → Revise web code

Refinement: Adjust style, format, and colors

Best Practice (Separate Parts): Keep physics/modeling code separate from UI code so the AI can handle them independently.

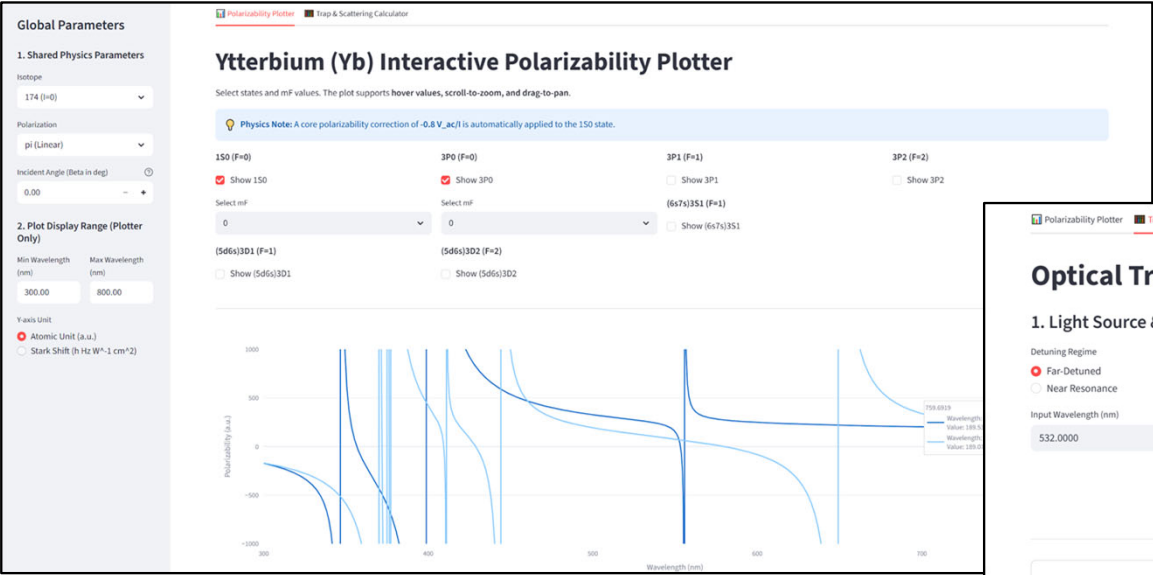




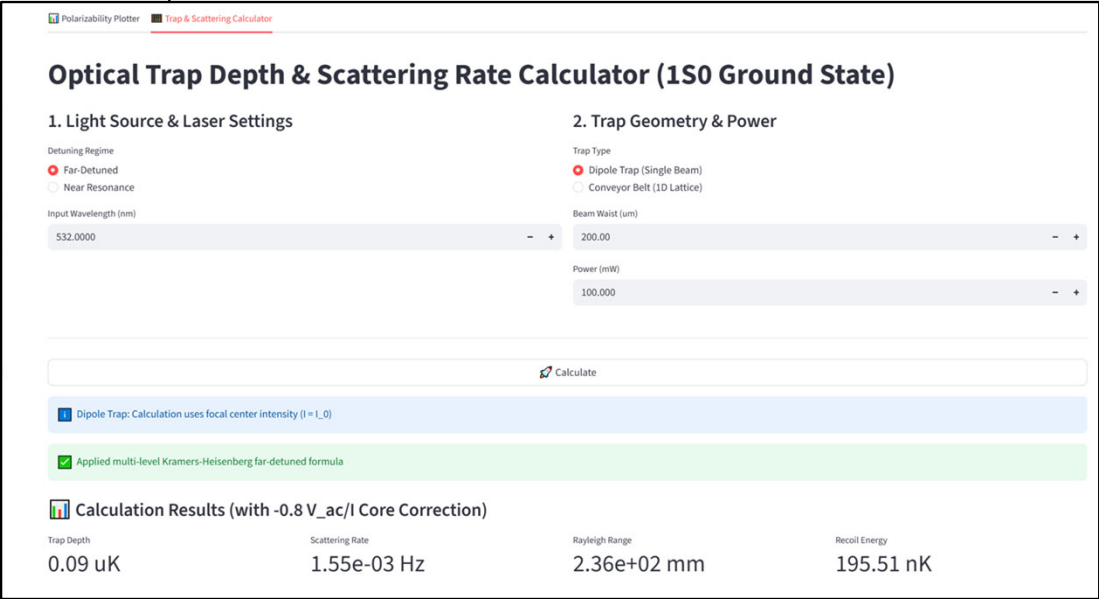
Result: Ytterbium properties web calculator



<https://yb-polarizability-calculator-tb3yemhxkfbttaqt3y5b5l.streamlit.app/>



New way to answer
PI's questions





Bonus: How to make a cartoon for your talk



Prompt:

i need a simple cartoon style picture of pulling an optical fiber into an optical nanofiber heating the center using an electric furnace.

